

[WS13] MVVM 2 - Repository

308-493 CROSS-PLATFORM MOBILE PROGRAMMING AND APPLICATIONS

สิ่งที่จะได้เรียนจาก workshop นี้

การสร้าง repository

การใช้งาน provider

[WS13-1] สร้าง Repository

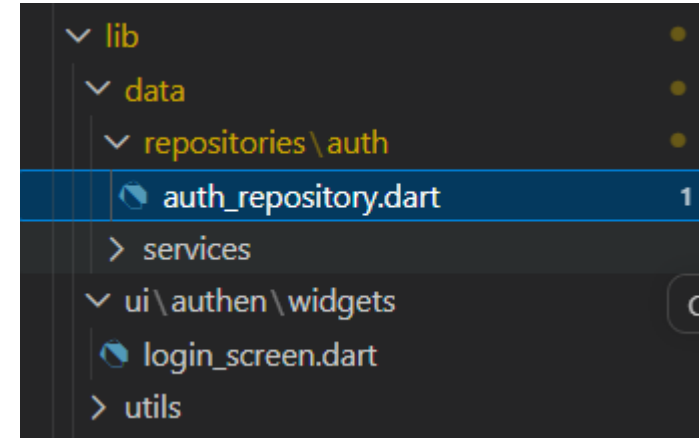
PATH 1

สร้าง Repository

สร้าง folder ชื่อ repositories ใน /lib/data

สร้าง folder ชื่อ auth ใน /lib/data/repositories

สร้าง file ชื่อ auth_repository.dart ใน
/lib/data/repositories



สร้าง Repository

[/lib/data/repositories/auth/auth_repository.dart](#)

สร้าง class ชื่อ AuthRepository ตาม
ภาพประกอบ

เราจะใช้ Repository ในการเรียกใช้งาน
class ในชั้น service

```
class AuthRepository extends ChangeNotifier {  
  bool? _isAuthenticated;  
  String? _authToken;  
}
```

สร้าง Repository

[/lib/data/repositories/auth/auth_repository.dart](#)

เราจะใช้ `_isAuthenticated` สำหรับเก็บข้อมูลว่า ผู้ใช้ได้ผ่านการยืนยันตัวตนหรือยัง ซึ่งมีข้อดีคือ เราไม่จำเป็นต้องดึง token ที่เก็บใน device ซ้ำ

```
class AuthRepository extends ChangeNotifier {  
  bool? _isAuthenticated;  
  String? _authToken;  
}
```

สร้าง Repository

[/lib/data/repositories/auth/auth_repository.dart](#)

สร้าง Constructor โดยรับตัวแปรเข้า 2 ตัว

authApiClient สำหรับเรียกใช้ method ต่าง ๆ จาก AuthRepository

sharedPreferencesService สำหรับเรียกใช้ method ที่เขียน/อ่านข้อมูลที่เก็บใน device

```
AuthRepository({  
  required AuthApiClient authApiClient,  
  required SharedPreferencesService sharedPreferencesService,  
}) : _authApiClient = authApiClient,  
     _sharedPreferencesService = sharedPreferencesService;  
  
final AuthApiClient _authApiClient;  
final SharedPreferencesService _sharedPreferencesService;
```

สร้าง Repository

[/lib/data/repositories/auth/auth_repository.dart](#)

เพิ่ม `_fetch()` เพื่อดึงข้อมูล
token จาก device

```
Future<void> _fetch() async {  
  final result = await _sharedPreferencesService.fetchToken();  
  
  switch (result) {  
    case Ok<String?>():  
      _authToken = result.value;  
      _isAuthenticated = result.value != null;  
      break;  
    case Error<String?>():  
      print("Error on _fetch");  
  }  
}
```

สร้าง Repository

[/lib/data/repositories/auth/auth_repository.dart](#)

สร้าง method สำหรับดึงข้อมูล (get) ไปใช้ใน code ภายนอก

```
Future<bool> get isAuthenticated async {  
  if (_isAuthenticated != null) {  
    return _isAuthenticated!;  
  }  
  
  await _fetch();  
  return _isAuthenticated ?? false;  
}
```

สร้าง Repository

/lib/data/repositories/auth/
auth_repository.dart

เพิ่ม _accessRequest สำหรับ
ขอ access token จาก server

```
Future<Result<AccessResponse>> _accessRequest(  
  String username,  
  String password,  
  String authToken,  
) async {  
  try {  
    String passwordEncode = sha256.convert(utf8.encode(password)).toString();  
    String combinedString = "$username&$passwordEncode&$authToken";  
    String authenSignature = sha256  
      .convert(utf8.encode(combinedString))  
      .toString();  
  
    final accessResponse = await _authApiClient.accessRequest(  
      AccessRequest(  
        authenSignature: authenSignature,  
        authToken: authToken,  
      ),  
    );  
  
    switch (accessResponse) {  
      case Ok<AccessResponse>():  
        return accessResponse;  
      case Error<AccessResponse>():  
        return Result.error(accessResponse.error);  
    }  
  } on Exception catch (error) {  
    return Result.error(error);  
  }  
}
```

สร้าง Repository

/lib/data/repositories/auth/
auth_repository.dart

เพิ่ม _authenRequest สำหรับ
อนุญาตเข้าสู่ระบบ

```
Future<Result<AccessResponse>> _authenRequest(  
  String username,  
  String password,  
) async {  
  try {  
    DateTime now = DateTime.now();  
    String formattedDateString = DateFormat('dd-MM-yyyy').format(now);  
    String combinedString = "$username&$formattedDateString";  
  
    String authenRequestString = sha256  
      .convert(utf8.encode(combinedString))  
      .toString();  
  
    final authenResponse = await _authApiClient.authenRequest(  
      AuthenRequest(authenRequest: authenRequestString),  
    );  
  
    switch (authenResponse) {  
      case Ok<AuthenResponse>():  
        return await _accessRequest(  
          username,  
          password,  
          authenResponse.value.authenToken,  
        );  
      case Error<AuthenResponse>():  
        return Result.error(authenResponse.error);  
    }  
  } on Exception catch (error) {  
    return Result.error(error);  
  }  
}
```

สร้าง Repository

/lib/data/repositories/auth/
auth_repository.dart

สร้าง login เพื่อให้สามารถเรียกใช้
จากภายนอก class ได้

```
Future<Result<void>> login({
  required String username,
  required String password,
}) async {
  try {
    final accessResponse = await _authenRequest(username, password);

    switch (accessResponse) {
      case Ok<AccessResponse>():
        await _sharedPreferencesService.saveToken(
          accessResponse.value.accessToken,
        );
        await _sharedPreferencesService.saveImageUrl(
          accessResponse.value.imageUrl,
        );
        await _sharedPreferencesService.saveUsername(username);

        return Result.ok(null);
      case Error<AccessResponse>():
        return Result.error(accessResponse.error);
    }
  } finally {
    notifyListeners();
  }
}
```

[WS13-2] ใช้งาน Provider

PATH 2

ทำไมต้องใช้ provider

สังเกตที่ constructor

เรารับตัวแปรเข้ามา 2 ตัว ซึ่งเป็น class สำหรับติดต่อกับ API

การทำแบบนี้จะทำให้เรา Unit testing ได้ง่ายขึ้น

เราจะใช้ Provider สำหรับตั้งค่าตัวแปรทั้งสอง

```
AuthRepository({  
  required AuthApiClient authApiClient,  
  required SharedPreferencesService sharedPreferencesService,  
}) : _authApiClient = authApiClient,  
    _sharedPreferencesService = sharedPreferencesService;  
  
final AuthApiClient _authApiClient;  
final SharedPreferencesService _sharedPreferencesService;
```

Provider

เราใช้ Provider เพื่อจัดการสถานะ (State Management)

ChangeNotifier (ผู้ผลิตข้อมูล)

- เมื่อข้อมูลเปลี่ยนแปลงจะใช้คำสั่ง `notifyListeners` เพื่อบอกให้ UI รู้

ChangeNotifierProvider (ผู้กระจายข้อมูล)

- ใช้ ChangeNotifier ในการกระจายข้อมูลให้ UI เรียกใช้งาน

Consumer (ผู้บริโภคข้อมูล)

- การดึงข้อมูลของ UI
- `context.read<T>()` เรียกใช้ method
- `context.watch<T>()` ดึงข้อมูลมาแสดง

```
flutter pub add provider
```

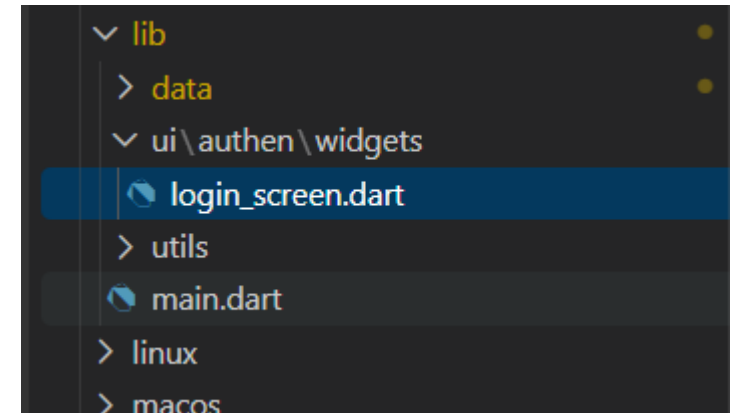
สร้าง LoginScreen

สร้าง folder ชื่อ ui ใน /lib

สร้าง folder ชื่อ authen ใน /lib/ui

สร้าง folder ชื่อ widgets ใน /lib/ui/authen

สร้างไฟล์ชื่อ login_screen.dart ใน
/lib/ui/authen



สร้าง LoginScreen

[/lib/authen/widgets/login_screen.dart](#)

สร้าง LoginScreen โดยกำหนดให้เป็น
StatelessWidget

```
class LoginScreen extends StatelessWidget {  
  const LoginScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container();  
  }  
}
```

สร้าง LoginScreen

/lib/authen/widgets/login_screen.dart

แก้ไข build ตามภาพประกอบ

```
@override
Widget build(BuildContext context) {
  final usernameController = TextEditingController();
  final passwordController = TextEditingController();

  return Scaffold(
    appBar: AppBar(title: const Text('Login')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          TextField(
            controller: usernameController,
            decoration: const InputDecoration(labelText: 'Username'),
          ), // TextField
          TextField(
            controller: passwordController,
            decoration: const InputDecoration(labelText: 'Password'),
            obscureText: true,
          ), // TextField
          const SizedBox(height: 20),
        ],
      ),
    ),
  );
}
```

สร้าง LoginScreen

/lib/authen/widgets/login_screen.dart

เพิ่มปุ่ม Sign In

```
ElevatedButton(  
  onPressed: () async {  
    final authRepo = context.read<AuthRepository>();  
  
    final result = await authRepo.login(  
      username: usernameController.text,  
      password: passwordController.text,  
    );  
  
    if (context.mounted) {  
      switch (result) {  
        case Ok():  
          ScaffoldMessenger.of(context).showSnackBar(  
            const SnackBar(content: Text('เข้าสู่ระบบสำเร็จ!')),  
          );  
          // TODO  
          break;  
        case Error():  
          print("${result.error}");  
          ScaffoldMessenger.of(context).showSnackBar(  
            SnackBar(  
              content: Text('เกิดข้อผิดพลาด: ${result.error}'),  
            ), // SnackBar  
          );  
          break;  
      }  
    }  
  },  
  child: const Text('Sign In'),  
), // ElevatedButton  
,  
, // Column  
, // Padding  
); // Scaffold  
}
```

กำหนด provider

[/lib/main.dart](#)

แก้ไข main ตามภาพประกอบ

```
Run | Debug | Profile
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => AuthRepository(
        authApiClient: AuthApiClient(
          host: "10.0.2.2",
          port: 3000,
        ), // AuthApiClient
        sharedPreferencesService: SharedPreferencesService(),
      ), // AuthRepository
      child: const MyApp(),
    ), // ChangeNotifierProvider
  );
}
```

กำหนด provider

[/lib/main.dart](#)

แก้ไข MyApp ตามภาพประกอบ
ทดสอบโปรแกรมบน android
emulator

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  
        colorScheme: .fromSeed(seedColor: Colors.deepPurple),  
      ), // ThemeData  
      home: const LoginScreen(),  
    ); // MaterialApp  
  }  
}
```

ฝึกทำ

ให้นักศึกษาสร้าง service และ repository สำหรับข้อมูล activity (ตาม function การทำงานใน node.js)